

**Hewlett Packard
Enterprise**

HPE CRAY EX USER TRAINING CUSTOMIZED FOR MPP LABS

Using PBS Pro with PALS

June 30, 2026

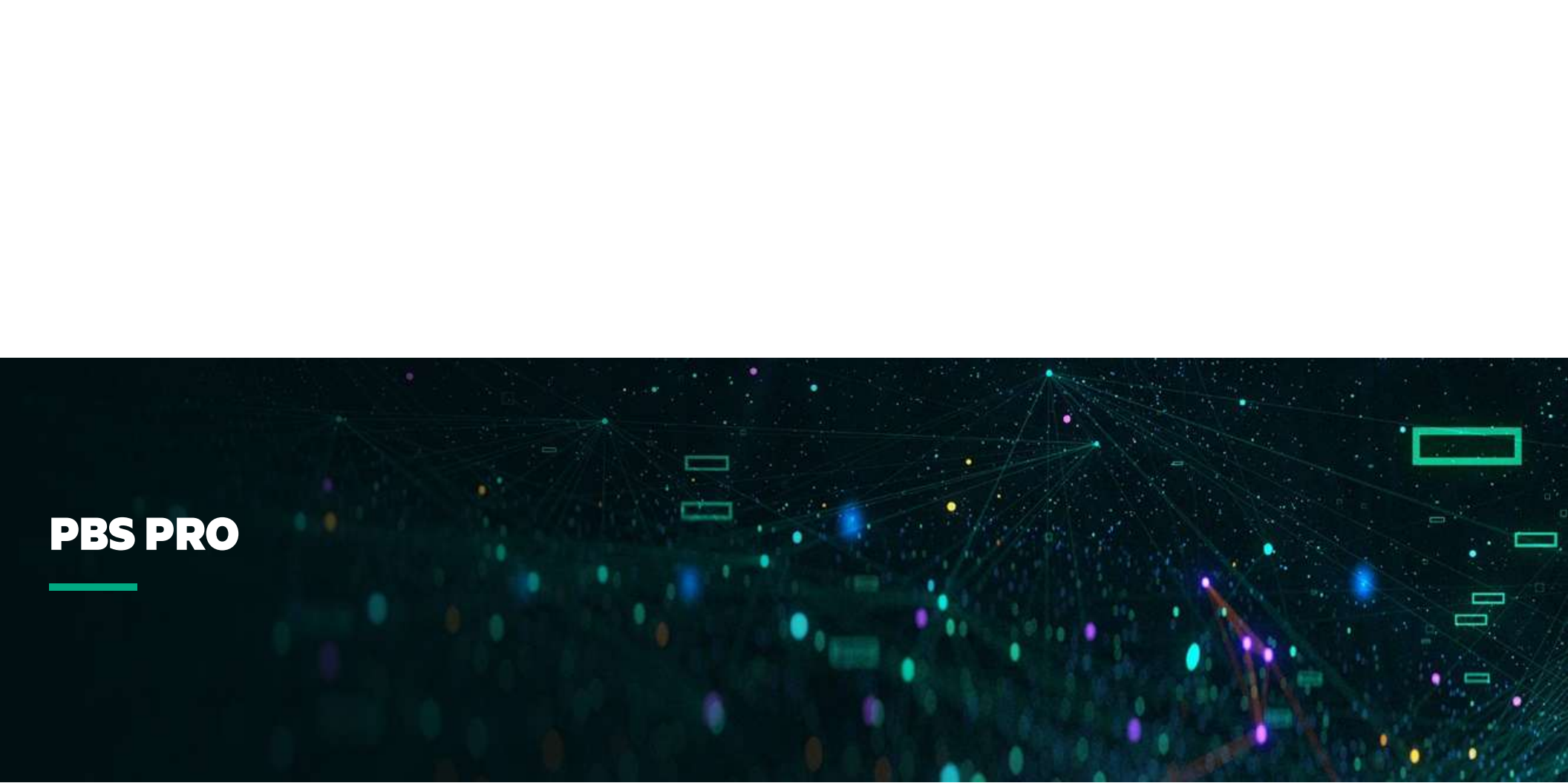


LEARNING OBJECTIVES



- After completing this module, you should be able to:
 - Launch a batch job on an HPE Cray EX Series system using PBS Pro
 - Check the status of running jobs with PBS Pro
 - Run a parallel application with the Parallel Application Launch Service





PBS PRO

PBS Pro terminology

- Job
 - Unit of work managed by PBS
 - A job is a related set of tasks, created and submitted by the user
- Job Array
 - Container for a collection of similar jobs submitted under a single job ID
- Task
 - Discrete software process with an individual address space that belongs to a job (e.g., MPI rank)
- Queue
 - Named container for jobs
- Node
 - Host or server running an operating system
- CPU
 - Software – Resource required to execute a program thread
 - Hardware – Core upon which computational tasks are executed
- Chunk
 - Set of resources allocated as a unit to a job
 - Specified inside a selection directive
 - All parts of a chunk come from the same host
 - Typically, there is one chunk per task



Common PBS Pro commands

PBS Pro Command	Description
<code>qsub</code>	Submit a job
<code>qsub -I</code>	Submit an interactive job
<code>qstat</code>	Check the status of job(s) queue(s) or the PBS Server
<code>qdel</code>	Delete a job
<code>qsig</code>	Send a signal to a job
<code>qhold JOB_ID</code>	Hold a job
<code>qrls JOB_ID</code>	Release a hold on a job
<code>pbsnodes</code>	Check the status of compute nodes
<code>tracejob JOB_ID</code>	Print log information about a job
<code>pbsdsh</code>	Distributed shell for PBS



Resources defined in PBS Pro

Resource Name	Description
host	Name of execution host
mem	Amount of physical memory i.e. workingset allocated to the job, either job-wide or host-level.
mpiprocs	Number of MPI processes for this chunk.
naccelerators	The number of accelerators (GPUs) on the host
ncpus	The number of cores or threads if Simultaneous Multi-Threading (SMT) is enabled
ompthreads	The number of OpenMP threads for this chunk.
walltime	Establishes a job-wide resource limit for the amount of wall-clock time

These resources can be specified for each chunk using `-l select` option

For additional details see the `qsub` man page or the PBS Pro Reference Guide

Chunk specification examples

- Reserve two chunks with 64 CPUs each

```
#PBS -l select=2:ncpus=64
```

- Reserve 10 chunks with 64 CPUs each on 10 different hosts

```
#PBS -l select=10:ncpus=64,place=scatter
```

- Use “scatter” = “to place each chunk on its own host”

- Reserve 10 chunks with 64 CPUs each to run 8 MPI ranks per chunk and 8 Open MP threads per rank

```
#PBS -l select=10:ncpus=64:mpiprocs=8:ompthreads=8
```

- `ompthreads` will set the environment variable `OMP_NUM_THREADS`

- Reserve 2 chunks with 64 CPUs each specifically on hosts `nid000006` and `nid000011`

```
#PBS -l select=1:ncpus=64:host=nid000006+1:ncpus=64:host=nid000011
```



Other common PBS options

- Combine `stderr` and `stdout` into one file

```
#PBS -j oe
```

- Limit the wallclock time a job is allowed to run to 15 minutes

```
#PBS -l walltime=00:15:00
```

- Export current environment to PBS job

```
#PBS -V
```

- Don't start job until after job *JOBID* has finished

```
#PBS -W depend=afterany:JOBID
```

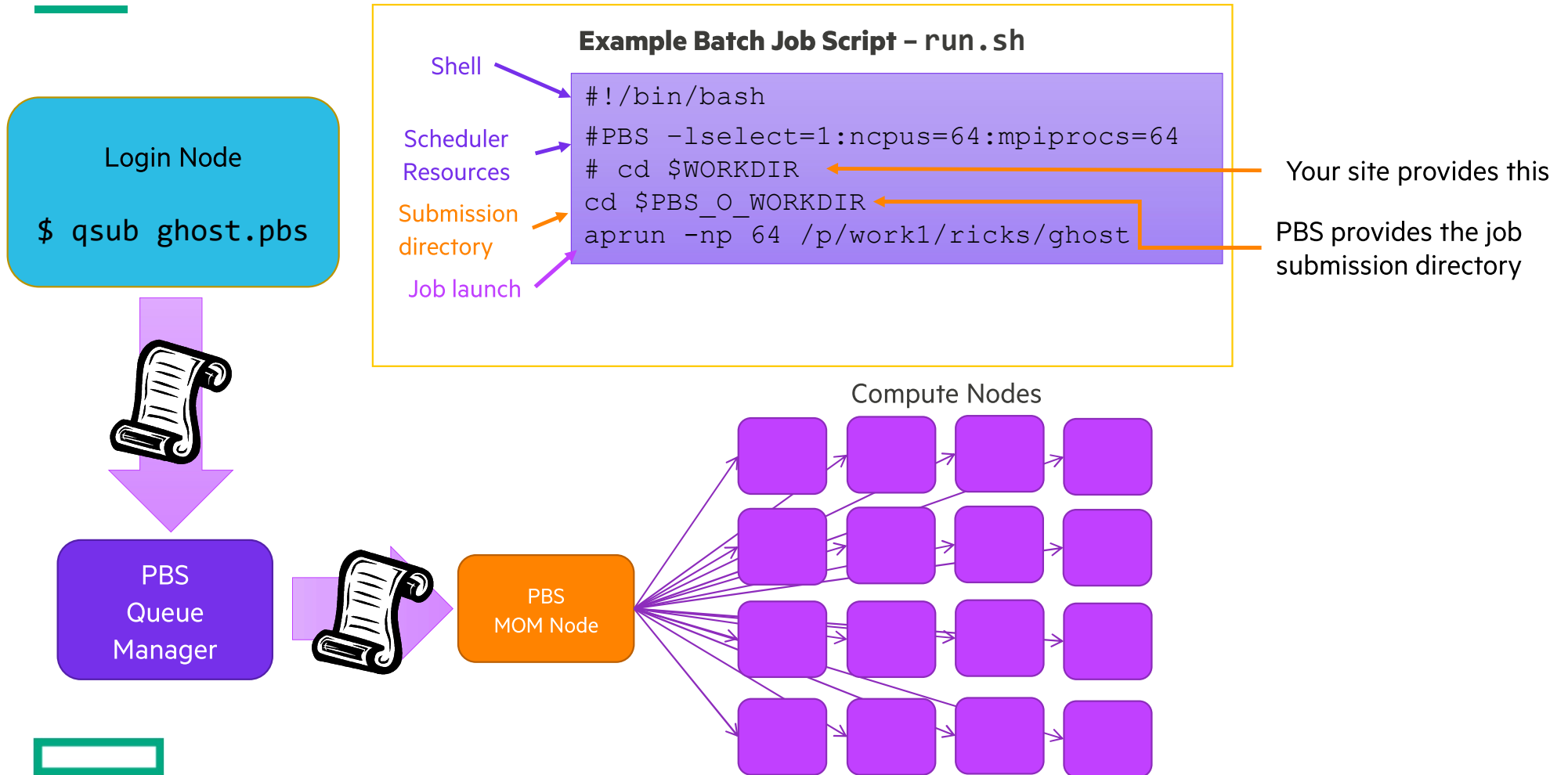
- Don't start job until 11:00 pm on December 24

```
#PBS -a 12242300
```

- Specifying century, year, month, day, and seconds are optional (See the user guide for default logic)



RUNNING A BATCH JOB



Sample job script with #PBS directives

```
user@uan1:~> cat run_hello.sh
#!/bin/bash
#PBS -l select=4:ncpus=64
#PBS -j oe
#PBS -q workq
#PBS -l walltime=00:02:00
```

#PBS directives specify:

- Requesting 4 “chunks” each with at least 64 logical cores
- Standard out and standard error should be consolidated
- This job should run in the “workq” queue
- This job has a max run time of two minutes

```
cd $PBS_O_WORKDIR
```

```
export OMP_NUM_THREADS=4
```

8 total MPI processes

4 MPI processes per compute node

```
aprun -n 8 -N 4 --cc depth -d 4 ./hellope | sort
```

Each MPI processes uses 4 OpenMP threads

```
user@uan1:~> qsub run_hello.sh
4009.uan1
```

```
user@uan1:~> head run_hello.sh.o4009
```

```
Hello from rank 0, thread 0, on xd225-01. (core affinity = 0)
Hello from rank 0, thread 1, on xd225-01. (core affinity = 1)
Hello from rank 0, thread 2, on xd225-01. (core affinity = 2)
Hello from rank 0, thread 3, on xd225-01. (core affinity = 3)
Hello from rank 1, thread 0, on xd225-01. (core affinity = 4)
Hello from rank 1, thread 1, on xd225-01. (core affinity = 5)
Hello from rank 1, thread 2, on xd225-01. (core affinity = 6)
Hello from rank 1, thread 3, on xd225-01. (core affinity = 7)
Hello from rank 2, thread 0, on xd225-01. (core affinity = 8)
Hello from rank 2, thread 1, on xd225-01. (core affinity = 9)
```

aprun is an MPI launcher
provided by PALS
It is an alternative to **mpiexec**

Common options for qstat

PBS Command	Description
qstat	Print job status using default format
qstat -f	Print job status of all jobs using long format
qstat -f JOB_ID	Print job status of JOB_ID in long format
qstat -a	Print job status using alternat format
qstat -as	Print job status with alternate format and a comment line for each job
qstat -u USER_NAME	Print job status for all jobs owned by the specified user
qstat - Qf	Print queue information using long format
qstat -q	Print queue information using alternate format
qstat -B	Print PBS Server status

Batch job status

```
training102@uan01:~> qstat -aw
```

pbs-service-nmn:

Job ID	Username	Queue	Jobname	SessID	NDS	TSK	Req'd Memory	Req'd Time	S	Elap Time
1113.pbs-service-nmn	training102	workq	hi.pbs	--	4	384	--	00:10	W	--
1114.pbs-service-nmn	training101	workq	big_hi.pbs	51051	10	960	--	00:12	R	00:03
1115.pbs-service-nmn	training101	workq	hey.pbs	--	2	128	--	00:10	Q	--

Status (S)	Description
E	Job is exiting after having run (if stays in E too long use qdel -Wforce)
H	H - Job is held
Q	job is queued, eligible to run or routed
R	Job is running
F	Job is finished
S	Job is suspended
W	job is waiting for its start time (-a option) to be reached



PBS Queues on a typical system

```
[ricks@breakpoint02 [ ] * UNCLASSIFIED * ~]$ qstat -q
```

```
server: pbs-0001
```

Queue	Memory	CPU Time	Walltime	Node	Run	Que	Lm	State
workq	--	--	--	--	0	0	--	E S
staff	--	--	24:00:00	--	0	28	--	E R
high	--	--	168:00:0	--	0	0	--	E S
interactive	--	--	12:00:00	--	0	0	--	E S
cots	--	--	96:00:00	--	0	0	--	E S
debug	--	--	01:00:00	--	0	2	--	E R
HIE	--	--	24:00:00	--	0	0	--	E R
frontier	--	--	168:00:0	--	0	0	--	E S
transfer	--	--	48:00:00	--	0	0	--	E S
standard	--	--	168:00:0	--	0	52	--	E R
standard-long	--	--	200:00:0	--	0	0	--	E S
background	--	--	24:00:00	--	0	0	--	E S
validation	--	--	--	--	0	0	--	D S
					0	82		

```
[ricks@breakpoint02 [ ] * UNCLASSIFIED * ~]$
```

qstat Listing with job comments

```
cmarsh@nid000004:~> qstat -as
```

```
pbs-service-nmn:
```

Job ID	Username	Queue	Jobname	SessID	NDS	TSK	Req'd Memory	Req'd Time	Elap S	Time
1010.pbs-servi*	cmarsh	workq	test.pbs	--	2	2	--	00:05	R	--
Job run at Mon Oct 18 at 15:25 on (nid000001:ncpus=1)+(nid000002:ncpus=1)										
1011.pbs-servi*	cmarsh	workq	test.pbs	--	2	2	--	00:05	R	--
Job run at Mon Oct 18 at 15:30 on (nid000001:ncpus=1)+(nid000002:ncpus=1)										
1017.pbs-servi*	cmarsh	workq	STDIN	49539	2	2	--	--	R	00:00
Job run at Mon Oct 18 at 16:45 on (nid000004:ncpus=1)+(nid000003:ncpus=1)										

-a implies all jobs

-s adds the comment line after each job listing

pbsnodes **Node listing**

pbsnodes provides detailed information about the nodes

The `-aS` option shows all nodes in a table with their attributes

```
cmarsh@uan01:/lus/cls01090/cmarsh> pbsnodes -a
nid000001
  Mom = nid000001
  ntype = PBS
  state = free
  pcpus = 256
  jobs = 1010.pbs-service-nmn/0, 1011.pbs-service-nmn/1
  resources_available.arch = linux
  resources_available.host = nid000001
  resources_available.mem = 263542264kb
  resources_available.ncpus = 256
  resources_available.vnode = nid000001
  resources_assigned.accelerator_memory = 0kb
  resources_assigned.hbmem = 0kb
  resources_assigned.mem = 0kb
  resources_assigned.naccelerators = 0
  resources_assigned.ncpus = 2
  resources_assigned.vmem = 0kb
  resv_enable = True
  sharing = default_shared
  license = 1
  last_state_change_time = Fri Oct 15 16:53:57 2021
  last_used_time = Fri Oct 15 16:53:57 2021
```

```
cmarsh@uan01:/lus/cls01090/cmarsh> pbsnodes -aS
```

vnode	state	OS	hardware	host	queue	mem	ncpus	nmics	ngpus	comment
nid000001	free	--	--	nid000001	--	251gb	256	0	0	--
nid000002	free	--	--	nid000002	--	251gb	256	0	0	--
nid000003	free	--	--	nid000003	--	251gb	256	0	0	--
nid000004	free	--	--	nid000004	--	251gb	256	0	0	-



PALS

Parallel Application Launch Service (PALS)

- PALS is the next iteration of ALPS (Application Level Placement Scheduler)
- Application launcher that enables PBS Pro to function normally
- Launch daemon (`palsd`) integrates with WLMs that have a compute node presence
 - E.g., PBS Pro's MoM
- Runs alongside the PBS MoM daemon on the compute node
- Coordinates execution of parallel applications on multiple compute nodes
 - Treats these as a unit rather than separate processes
- Integrates with Keycloak in CSM systems for user authentication (AuthN) and authorization (AuthZ)



PALS commands

Option	Purpose
<code>mpiexec</code>	Launch an application using a common MPI style interface
<code>aprun</code>	Launch an application using a Cray XC style interface
<code>palsig</code>	Signal the application using PALS
<code>palstat</code>	Check an application's status using PALS
<code>palsctrl</code>	Control or query a set of PALS daemons
<code>palscp</code>	Transfer files to compute nodes
<code>palscmd</code>	Run helper processes on compute nodes

aprun **Command line options**

Option	Shorthand	Purpose
-pes	-n	The number of processing elements (i.e., ranks) to run
-cpus-per-pe	-d	The number of cpus (cores) to be allocated per process. This may be useful if the job is multithreaded and requires more than one cpu per task for optimal performance.
-pes-per-node	-N	Request that a specific number or ranks per node be allocated to this job.
-memory-per-pe	-m	Request a certain amount of memory per processing element
-node-list	-L	List of node(s) where task should run
-cpu_binding	-cc	Controls how tasks are bound to cores. Options include: • cpu – Bind ranks to a single thread (default) • depth – Bind ranks to the number of cpus specified with -d • numa_node - Bind ranks each thread in its assigned NUMA node • core - Bind ranks to every thread • none - No CPU binding

mpirexec Command line options

Option	Shorthand	Purpose
--np	-n	The number of processing elements (i.e., ranks) to run
--depth		The number of cpus (cores) to be allocated per process. This may be useful if the job is multithreaded and requires more than one cpu per task for optimal performance.
--ppn	n/a	Request that a specific number or ranks per node be allocated to this job.
--hostlist	-hosts	Comma delimited list of node(s) where task should run
--hostfile	-f	File containing a list of node(s) where task should run
--cpu_bind	n/a	Controls how tasks are bound to cores. Options include: • none - No CPU binding • numa, socket, core, thread - Bind ranks to the specified hardware • depth - Bind ranks to number of threads in 'depth' argument • list - Bind ranks to colon-separated rangelists of CPUs • mask - Bind ranks to comma-separated bitmasks of CPUs
--mem-bind	n/a	Controls which numa node a core has access to. Options include none, local, list, and mask

PALS environment variables

Environment Variable	Purpose
PALS_APID	Unique application identifier
PALS_RANKID	Application rank identifier
PALS_LOCAL_RANKID	Application rank identifier within a node
PALS_DEPTH	Number of CPUs per rank for the application
PALS_NODEID	Application relative node identifier

Additional variables are documented in the **aprun** and **mpiexec** man pages

PALS environment variables – test job

```
styx-login1:~ # qsub -I -l select=4:ncpus=128
qsub: waiting for job 4031.styx-login1 to start
qsub: job 4031.styx-login1 ready
```

This example references PALS environment variables in a 4 node interactive job

```
xd225-01:~ # module load cray-pals
xd225-01:~ # aprun -n 4 -N 1 bash -c 'echo " PALS_RANKID: " $PALS_RANKID " HOSTNAME: " $(hostname) "
PALS_LOCAL_RANKID: " $PALS_LOCAL_RANKID " PALS_DEPTH: " $PALS_DEPTH " PALS_NODEID: " $PALS_NODEID' | sort
PALS_RANKID: 0 HOSTNAME: xd225-01 PALS_LOCAL_RANKID: 0 PALS_DEPTH: 1 PALS_NODEID: 0
PALS_RANKID: 1 HOSTNAME: xd225-02 PALS_LOCAL_RANKID: 0 PALS_DEPTH: 1 PALS_NODEID: 1
PALS_RANKID: 2 HOSTNAME: xd225-04 PALS_LOCAL_RANKID: 0 PALS_DEPTH: 1 PALS_NODEID: 2
PALS_RANKID: 3 HOSTNAME: xd225-05 PALS_LOCAL_RANKID: 0 PALS_DEPTH: 1 PALS_NODEID: 3
xd225-01:~ # aprun -n 4 -N 4 bash -c 'echo " PALS_RANKID: " $PALS_RANKID " HOSTNAME: " $(hostname) "
PALS_LOCAL_RANKID: " $PALS_LOCAL_RANKID " PALS_DEPTH: " $PALS_DEPTH " PALS_NODEID: " $PALS_NODEID' | sort
PALS_RANKID: 0 HOSTNAME: xd225-01 PALS_LOCAL_RANKID: 0 PALS_DEPTH: 1 PALS_NODEID: 0
PALS_RANKID: 1 HOSTNAME: xd225-01 PALS_LOCAL_RANKID: 1 PALS_DEPTH: 1 PALS_NODEID: 0
PALS_RANKID: 2 HOSTNAME: xd225-01 PALS_LOCAL_RANKID: 2 PALS_DEPTH: 1 PALS_NODEID: 0
PALS_RANKID: 3 HOSTNAME: xd225-01 PALS_LOCAL_RANKID: 3 PALS_DEPTH: 1 PALS_NODEID: 0
xd225-01:~ # mpiexec -n 4 --ppn 2 --depth 4 bash -c 'echo " Hostname: " $(hostname) " PALS_RANKID: "
$PALS_RANKID " PALS_LOCAL_RANKID: " $PALS_LOCAL_RANKID " PALS_NODEID: " $PALS_NODEID' | sort
Hostname: xd225-01 PALS_RANKID: 0 PALS_LOCAL_RANKID: 0 PALS_NODEID: 0
Hostname: xd225-01 PALS_RANKID: 1 PALS_LOCAL_RANKID: 1 PALS_NODEID: 0
Hostname: xd225-02 PALS_RANKID: 2 PALS_LOCAL_RANKID: 0 PALS_NODEID: 1
Hostname: xd225-02 PALS_RANKID: 3 PALS_LOCAL_RANKID: 1 PALS_NODEID: 1
```


Lab exercise – PBS Pro

45 minutes

- Lab exercises may be run on the MPP labs cluster or the HPE Training cluster Styx
 - Lab instructions are in the handout LG-M03_PBS_Pro.20260630
- PBS Pro lab exercises
 - Use pbs pro commands
 - Create and run batch job scripts
 - Run C and Python MPI applications
- Hint – to check which nodes a running job is using try:
 - `qstat -n JOB_ID`
 - `tracejob JOB_ID`